Feature engineering - categorical features

While it is ok to have your target vector in classification represent categories as strings for example, categorical features need to be transformed in *scikit-learn*.

There are two major types:

1. Ordinal use `OrdinalEncoder`
2. Nominal use `OneHotEncoder` (can handle non-string features, `DictVectorizer()` cannot)

Question: What is the difference between ordinal and nominal features? Can you give examples?

Answer: Ordinal has order, nominal does not. Ordinal would be `[small, medium, large]`, nominal would be `[apple, banana, orange]`

In [1]:
```
import pandas as pd
import mglearn
```

In [2]:
```
data = pd.DataFrame([
    {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},
    {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},
    {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},
    {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'}
])
```

In [3]:
```
data
```

Out[3]:

	price	rooms	neighborhood
0	850000	4	Queen Anne
1	700000	3	Fremont
2	650000	3	Wallingford
3	600000	2	Fremont

OrdinalEncoder

IMPORTANT: Neighborhoods are *nominal* and should not be encoded with an OrdinalEncoder. We do this here only for comparison to One-Hot Encoder.

In [4]:
```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder()
enc.fit(data[['neighborhood']])
```

Out[4]:
```
OrdinalEncoder
OrdinalEncoder()
```

In [5]:
```
enc.categories_
```

Out[5]:
```
[array(['Fremont', 'Queen Anne', 'Wallingford'], dtype=object)]
```

In [6]:
```
enc.transform(data[['neighborhood']])
```

Out[6]:
```
array([[1.],
       [0.],
       [2.],
       [0.]])
```

In [7]:
```
enc.inverse_transform([[1]])
```

Out[7]:
```
array([['Queen Anne']], dtype=object)
```

OneHotEncoder

In [8]:
```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False)
enc.fit(data[['neighborhood']])
```

Out[8]:
```
OneHotEncoder
OneHotEncoder(sparse_output=False)
```

In [9]:
```
enc.get_feature_names_out()
```

Out[9]:
```
array(['neighborhood_Fremont', 'neighborhood_Queen Anne',
       'neighborhood_Wallingford'], dtype=object)
```

In [10]:
```
enc.transform(data[['neighborhood']])
```

Out[10]:
```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

In [11]:
```
data_enc = pd.DataFrame(enc.transform(data[['neighborhood']]), columns=enc.get_feature
data_enc
```

Out[11]:

	neighborhood_Fremont	neighborhood_Queen Anne	neighborhood_Wallingford
0	0.0	1.0	0.0
1	1.0	0.0	0.0
2	0.0	0.0	1.0
3	1.0	0.0	0.0

Question: Can you add `data_enc` to the original `data`, dropping the `'neighborhood'` column?

In [12]:
```
data.head()
```

Out[12]:

	price	rooms	neighborhood
0	850000	4	Queen Anne
1	700000	3	Fremont
2	650000	3	Wallingford
3	600000	2	Fremont

In [13]:
```
# Answer here
data2 = pd.concat([data.drop(columns='neighborhood'), data_enc], axis=1)
data2
```

Out[13]:

	price	rooms	neighborhood_Fremont	neighborhood_Queen Anne	neighborhood_Wallingford
0	850000	4	0.0	1.0	0.0
1	700000	3	1.0	0.0	0.0
2	650000	3	0.0	0.0	1.0
3	600000	2	1.0	0.0	0.0

In practice, we would use `ColumnTransformer` in a pipeline. We will see this later.

One-hot encoding (cont'd)

We have already seen how one-hot encoding transforms a categorical feature column into multiple output columns containing 0's and 1's

In [14]:
```
# create a DataFrame with an integer feature and a categorical string feature
demo_df = pd.DataFrame({'Integer Feature': [0, 1, 2, 1],
                        'Categorical Feature': ['socks', 'fox', 'socks', 'box']})
display(demo_df)
```

	Integer Feature	Categorical Feature
0	0	socks
1	1	fox
2	2	socks
3	1	box

In [15]:
```
from sklearn.preprocessing import OneHotEncoder
# Setting sparse=False means OneHotEncode will return a numpy array, not a sparse matr
ohe = OneHotEncoder(sparse_output=False)
print(ohe.fit_transform(demo_df))
```

```
[[1. 0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 1. 0.]
 [0. 0. 1. 0. 0. 1.]
 [0. 1. 0. 1. 0. 0.]]
```

In [16]:
```
print(ohe.get_feature_names_out())
```

```
['Integer Feature_0' 'Integer Feature_1' 'Integer Feature_2'
 'Categorical Feature_box' 'Categorical Feature_fox'
 'Categorical Feature_socks']
```

One-hot encoding a large dataset

In [17]:
```
import os
# The file has no headers naming the columns, so we pass header=None
# and provide the column names explicitly in "names"
adult_path = os.path.join(mglearn.datasets.DATA_PATH, "adult.data")
data = pd.read_csv(
    adult_path,
    header=None,
    index_col=False,
    skipinitialspace=True, #remove space after comma
    names=['age', 'workclass', 'fnlwgt', 'education',  'education-num',
           'marital-status', 'occupation', 'relationship', 'race', 'gender',
           'capital-gain', 'capital-loss', 'hours-per-week', 'native-country',
           'income'])
# For illustration purposes, we only select some of the columns
data = data[['age', 'workclass', 'education', 'gender', 'hours-per-week',
             'occupation', 'income']]
# IPython.display allows nice output formatting within the Jupyter notebook
display(data.head())
```

	age	workclass	education	gender	hours-per-week	occupation	income
0	39	State-gov	Bachelors	Male	40	Adm-clerical	<=50K
1	50	Self-emp-not-inc	Bachelors	Male	13	Exec-managerial	<=50K
2	38	Private	HS-grad	Male	40	Handlers-cleaners	<=50K
3	53	Private	11th	Male	40	Handlers-cleaners	<=50K
4	28	Private	Bachelors	Female	40	Prof-specialty	<=50K

In [18]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   age             32561 non-null  int64 
 1   workclass       32561 non-null  object
 2   education       32561 non-null  object
 3   gender          32561 non-null  object
 4   hours-per-week  32561 non-null  int64 
 5   occupation      32561 non-null  object
 6   income          32561 non-null  object
dtypes: int64(2), object(5)
memory usage: 1.7+ MB
```

In [19]:
```
ohe = OneHotEncoder(sparse_output=False)
print(ohe.fit_transform(data['education'].values.reshape(-1,1)).shape)
```

```
(32561, 16)
```

One column goes in, 16 columns come out, meaning that the education column has 16 discrete values

In [20]:
```
print(ohe.get_feature_names_out(input_features=['education']))
```

```
['education_10th' 'education_11th' 'education_12th' 'education_1st-4th'
 'education_5th-6th' 'education_7th-8th' 'education_9th'
 'education_Assoc-acdm' 'education_Assoc-voc' 'education_Bachelors'
 'education_Doctorate' 'education_HS-grad' 'education_Masters'
 'education_Preschool' 'education_Prof-school' 'education_Some-college']
```